

00_SPIS_TRESCI

Spis tresci dokumentacji

Projekt: Automatyczne parkowanie w Unity 3D

Pliki dokumentacji

1. 01_Dokumentacja_techiczna.md

- pelny opis techniczny projektu,
- architektura,
- klasy,
- FSM,
- sensory,
- scenariusze,
- generator scen,
- ograniczenia.

2. 02_Jak_stworzyc_od_zera.md

- instrukcja stworzenia podobnego projektu krok po kroku,
- konfiguracja Unity,
- tworzenie auta,
- tworzenie sensorow,
- tworzenie FSM,
- tworzenie map,
- strojenie punktow parkowania.

3. 03_Testy_i_uruchomienie.md

- instalacja,
- uruchomienie,
- testy map 1, 2 i 3,
- kryteria zaliczenia,
- typowe problemy.

4. 04_Opis_do_obrony.md

- krotka sciaga do rozmowy z prowadzacym,
- odpowiedzi na pytania,
- opis map,
- opis uproszczen.

5. 05_Mapa_plikow_i_parametrow.md

- gdzie w kodzie zmienic predkosc,
- gdzie zmienic sensory,
- gdzie zmienic tor parkowania,

- gdzie zmienić czerwone auto,
- gdzie zmienić UI i HUD.

Najważniejsze do przeczytania przed obroną

Jeśli masz mało czasu:

1. Przeczytaj 04_Opis_do_obrony.md.
2. Przeczytaj sekcje o FSM w 01_Dokumentacja_techiczna.md.
3. Przeczytaj test mapy 3 w 03_Testy_i_uruchomienie.md.
4. Zapamiętaj, że finalny manewr działa deterministycznie "po szynach", a sensory i FSM służą do pokazania logiki automatycznego parkowania.

01_Dokumentacja_techiczna

Dokumentacja techniczna projektu

Projekt: Automatyczne parkowanie w Unity 3D

Wersja dokumentacji: 2026-06-15

Charakter projektu: deterministyczna symulacja parkowania, bez machine learningu

1. Cel projektu

Celem projektu jest pokazanie prostego systemu automatycznego parkowania samochodu w Unity.

Program ma:

- uruchamiać się w Unity jako zwykła scena 3D,
- mieć trzy scenariusze testowe,
- używać czujników opartych o Raycast i CheckBox,
- sterować autem przez maszyny stanów FSM,
- reagować na przeszkodę dynamiczną,
- pokazać wynik w czytelny sposób na ekranie,
- być możliwy do wytłumaczenia bez znajomości ML-Agents i bez trenowania modelu.

Projekt nie jest symulatorem profesjonalnej fizyki pojazdów. To wersja zaliczeniowa: stabilna, powtarzalna i prosta do omówienia. Najważniejsze jest pokazanie algorytmu, podziału na stany, sensorów i reakcji na sytuacje na parkingu.

2. Główna idea działania

Auto nie uczy się samo. Działa według deterministycznego scenariusza:

1. Auto jedzie powoli wzdłuż drogi.
2. Sensory mierzą odległości od przeszkód.
3. Program publikuje informacje o znalezionym miejscu parkingowym.

4. Maszyna stanow przechodzi przez kolejne etapy parkowania.
5. W mapach demonstracyjnych finalny manewr jest wykonywany po wyznaczonych punktach, jak po szynie.
6. Po dojechaniu do ostatniego punktu auto zatrzymuje sie w stanie Parked.

Finalna wersja celowo uzywa trybu "po szynach" dla manewru parkowania. Powod: w Unity prosta, niekonfigurowana fizyka pojazdu potrafi dawac niestabilne wyniki, a projekt ma dzialac przewidywalnie podczas prezentacji.

3. Struktura folderow

Po zainstalowaniu paczki w projekcie Unity wazne foldery sa takie:

[blok kodu / diagram w pliku Markdown]

Folder Documentation znajduje sie w paczce poza Assets, zeby nie mieszac dokumentow z logika Unity.

4. Najwazniejsze klasy

Plik Rola

--- ---

ParkingTypes.cs Definicje typow danych: stany FSM, typ parkowania, odczyty sensorow, kandydat miejsca.

ParkingCarController.cs Sterowanie pojazdem: predkosc, skret, hamowanie, ruch kinematyczny i ruch po punktach.

ParkingSensors.cs Czujniki Raycast, wykrywanie luk i walidacja wolnej przestrzeni.

ParkingStateMachine.cs Glowna logika FSM: skanowanie, walidacja, manewr, stop awaryjny, zaparkowanie.

ParkingScenario.cs Dane scenariusza: punkt akceptacji miejsca, lista punktow manewru, opis wykrytej luki.

MovingObstacle.cs Ruch czerwonego auta i wymuszenie zatrzymania niebieskiego auta.

ParkingDemoBuilder.cs Generator materialow, prefabu auta, UI i trzech scen testowych.

VehicleVisualRig.cs Wizualny obrot kol i skret przednich kol.

DebugHud.cs Tekst debug w lewym gornym rogu.

MapManager.cs Ladowanie scen i restart.

ImmediateMapUi.cs Zapasowe przyciski OnGUI do zmiany map.

CameraFollow.cs Kamera sledzaca auto.

5. Dane publiczne i struktury

ParkingState

Enum opisuje aktualny stan automatu:

[blok kodu / diagram w pliku Markdown]

Znaczenie stanów:

- Scan - auto jedzie i szuka miejsca.
- ValidateSpot - program potwierdza znalezione miejsce.
- Positioning - auto ustawia sie do manewru.
- ReverseTurn - pierwszy skret przy wjeździe w miejsce.
- CounterTurn - kontra, czyli drugi etap ustawiania auta.
- Straighten - prostowanie auta w miejscu.
- Parked - koniec.
- EmergencyStop - zatrzymanie awaryjne, np. przy przeszkodzie dynamicznej.

ParkingType

[blok kodu / diagram w pliku Markdown]

Perpendicular oznacza parkowanie prostopadle, Parallel oznacza parkowanie równolegle.

SensorReadings

Struktura przechowuje odległości z sensorów:

[blok kodu / diagram w pliku Markdown]

Przykład: rightFront to odległość po prawej stronie z przodu pojazdu, front to odległość przed pojazdem.

ParkingSpotCandidate

Opisuje kandydat na miejsce parkingowe:

[blok kodu / diagram w pliku Markdown]

W scenach demonstracyjnych kandydat jest częściowo publikowany z ParkingScenario, żeby prezentacja była powtarzalna.

6. Maszyna stanów FSM

FSM znajduje się w ParkingStateMachine.cs. To najważniejszy skrypt projektu.

Schemat:

[blok kodu / diagram w pliku Markdown]

Awake

W Awake pobierane sa komponenty:

- ParkingCarController,
- ParkingSensors,
- ParkingManeuverPlanner,
- opcjonalnie ParkingScenario.

Jezeli obiekt ma ParkingScenario, to:

[blok kodu / diagram w pliku Markdown]

Oznacza to, ze ruch auta bedzie sterowany przez punkty scenariusza, a nie przez swobodna fizyke.

FixedUpdate

FixedUpdate dziala w stalym kroku fizycznym Unity. W projekcie jest to dobre miejsce na logike ruchu auta.

Najpierw aktualizowany jest czas stanu:

[blok kodu / diagram w pliku Markdown]

Potem sprawdzany jest stop awaryjny:

- raycastEmergency - przeszkoda z sensorow,
- scriptedEmergency - przeszkoda dynamiczna wymuszona przez MovingObstacle.

Jesli auto jest w trakcie manewru i pojawia sie przeszkoda, FSM przechodzi do EmergencyStop.

UpdateScenarioScan

W scenach demonstracyjnych auto nie czeka na przypadkowy wynik sensorow, tylko dojezdza do punktu scanAcceptPoint.

Kiedy dojedzie:

1. budowany jest ParkingSpotCandidate,
2. kandydat jest wyświetlany w HUD,
3. FSM przechodzi do ValidateSpot.

UpdateScenarioValidation

Auto zatrzymuje się na krótką pauzę:

[blok kodu / diagram w pliku Markdown]

Potem rozpoczyna pierwszy krok scenariusza.

UpdateScenarioManeuverState

To metoda, która wykonuje punkty manewru:

1. Pobiera aktualny ParkingScenarioStep.
2. Jeśli krok ma smoothArc = true, używa UpdateSmoothScenarioStep.
3. Jeśli nie, używa prostego MoveToPose.
4. Po zakończeniu kroku przechodzi do kolejnego.

Nazwa smoothArc została historycznie, ale w finalnej wersji ruch jest celowo zrobiony "po szynach": prosta interpolacja pozycji między punktami i łagodna interpolacja kąta.

UpdateSmoothScenarioStep

Finalna wersja działa tak:

[blok kodu / diagram w pliku Markdown]

Na starcie kroku zapamiętujemy:

- pozycję startową,
- kąt startowy,
- różnicę kąta do punktu docelowego,
- długość odcinka.

Potem w każdym kroku:

[blok kodu / diagram w pliku Markdown]

To znaczy:

- stepProgress rośnie od 0 do 1,
- Vector3.Lerp przesuwa auto po prostej szynie,
- Mathf.SmoothStep łagodzi start i koniec obrotu,
- Mathf.DeltaAngle gwarantuje najkrótszy obrot, bez 360 stopni.

Krok kończy się, gdy:

[blok kodu / diagram w pliku Markdown]

Nie ma już dokrecania w miejscu. To ważne, bo wcześniejsze wersje potrafiły stać i obracać samochód.

7. Sterowanie autem

Sterowanie znajduje się w ParkingCarController.cs.

Dwa tryby pracy

1. Tryb kinematyczny:

- używany, gdy nie ma ParkingScenario,
- auto reaguje na SetControls(throttle, steering, brake),
- obrot liczony jest uproszczonym modelem rowerowym.

2. Tryb po punktach:

- używany w scenach demonstracyjnych,
- UseExternalPoseControl = true,
- FSM ustawia pozycje przez SetPoseAlongPath.

SetControls

[blok kodu / diagram w pliku Markdown]

Parametry:

- throttle od -1 do 1, ujemny oznacza cofanie,
- steering od -1 do 1, kierunek skretu,
- brake od 0 do 1.

Kinematyka w FixedUpdate

Predkosc nie zmienia się natychmiast. Używany jest Mathf.MoveTowards, czyli płynne dojście do predkosci docelowej.

Skret tez nie zmienia sie od razu:

[blok kodu / diagram w pliku Markdown]

Obrot auta w trybie kinematycznym:

[blok kodu / diagram w pliku Markdown]

To uproszczony model rowerowy. Zaklada, ze pojazd ma jeden wirtualny przod i jeden wirtualny tyl.

SetPoseAlongPath

Ta metoda jest kluczowa dla finalnego trybu:

[blok kodu / diagram w pliku Markdown]

Ustawia:

- pozycje auta,
- rotacje tylko po osi Y,
- aktualna predkosc do HUD i kol,
- odpowiedz skretu dla wizualnych kol.

Funkcja FlattenYaw zeruje niepotrzebne obroty w osiach X i Z, zeby auto nie przechylalo sie ani nie obracalo przypadkowo.

8. Sensory

Sensory sa w ParkingSensors.cs.

Odczyty Raycast

Metoda GetReadings wykonuje kilka promieni:

- przod,
- tyl,
- prawa strona: przod, srodek, tyl,
- lewa strona: przod, srodek, tyl.

Promienie sa rysowane w Scene View:

- czerwony oznacza trafienie w przeszkodę,
- zielony oznacza brak przeszkody.

Najważniejsza metoda pomocnicza:

[blok kodu / diagram w pliku Markdown]

Zwraca odległość do przeszkody albo maksymalny zasięg.

Wykrywanie luki

TryFindSpot sprawdza, czy po prawej stronie jest wolna przestrzeń:

[blok kodu / diagram w pliku Markdown]

Jeśli wolna przestrzeń trwa odpowiednio długo, powstaje kandydat na miejsce.

Walidacja objętości

Sama odległość z promieni nie wystarczy. Dlatego jest dodatkowy test:

[blok kodu / diagram w pliku Markdown]

CheckBox sprawdza, czy w przewidywanym miejscu parkingowym nie ma obiektu. To zabezpiecza przed fałszywą luką.

9. Scenariusze parkowania

Scenariusz jest w ParkingScenario.cs.

Najważniejsze pola:

- parkingType - prostopadłe albo równoległe,
- scanAcceptPoint - gdzie auto uznaje miejsce za znalezione,
- scanSpeed - prędkość podczas dojazdu,
- validatePause - pauza po znalezieniu miejsca,
- spotStart, spotEnd, spotWidth - opis miejsca do HUD,
- steps - lista punktów manewru.

Pojedynczy krok:

[blok kodu / diagram w pliku Markdown]

W finalnej wersji najważniejsze są:

- state,
- target,
- reverse,
- speed,
- timeout.

curveHandle został jako pole kompatybilne z poprzednimi wersjami, ale aktualny ruch po szynach nie korzysta z krzywych Beziera.

10. Generator scen

ParkingDemoBuilder.cs jest skryptem edytorowym. Nie działa w buildzie jako logika gry, tylko generuje zasoby w Unity.

Menu:

[blok kodu / diagram w pliku Markdown]

Po kliknięciu wykonywane jest:

[blok kodu / diagram w pliku Markdown]

Ta metoda:

1. tworzy foldery,
2. tworzy materiały,
3. tworzy prefaby,
4. generuje menu główne,
5. generuje mapę 1,
6. generuje mapę 2,
7. generuje mapę 3,
8. ustawia sceny w Build Settings.

Materiały

CreateMaterials tworzy materiały:

- asfalt,

- beton,
- linie parkingowe,
- niebieski samochód,
- szare samochody,
- czerwony samochód dynamiczny,
- sciany,
- opony,
- szyby.

Model auta

AddVehicleVisual tworzy auto z prymitywów Unity:

- Body - główna bryła auta,
- Cabin - kabina/szyby,
- YellowFrontMarker albo FrontLights - oznaczenie przodu,
- 4 walce jako koła.

To nie jest realistyczny model 3D, ale pozwala czytelnie pokazać przód, tył i skręt.

11. Mapa 1

Nazwa: Map_01_Perpendicular

Założenie:

- parking prostokątny,
- większość miejsc zajęta,
- jedna luka jest za wąska,
- druga luka jest miejscem docelowym.

Auto startuje:

[blok kodu / diagram w pliku Markdown]

Miejsca i auta są po prawej stronie. Auto ma dojechać do punktu skanowania, zignorować złą lukę i zaparkować w miejscu docelowym.

Tor manewru jest zapisany w ConfigurePerpendicularScenario.

12. Mapa 2

Nazwa: Map_02_Parallel

Założenie:

- waska ulica,
- parkowanie rownolegle,
- auto ma wykonac cofanie i kontre,
- finalnie ma stac rownolegle do drogi.

Tor jest zapisany w ConfigureParallelScenario.

13. Mapa 3

Nazwa: Map_03_Dynamic

Zalozenie:

- scena podobna do mapy 1,
- miejsca parkingowe sa po lewej stronie,
- czerwone auto jedzie z naprzeciwka,
- niebieskie auto musi zaczekac,
- po przejechaniu czerwonego auta niebieskie parkuje po lewej.

Najwazniejsze punkty:

[blok kodu / diagram w pliku Markdown]

Finalne miejsce jest miedzy liniami:

- $z = -10.1$,
- $z = -6.7$.

Finalny punkt niebieskiego auta:

[blok kodu / diagram w pliku Markdown]

To srodek miejsca parkingowego.

14. Przeszkoda dynamiczna

MovingObstacle.cs przesuwa czerwone auto miedzy pointA i pointB.

W mapie 3:

- czerwone auto startuje z przodu,
- rusza, gdy niebieskie jest w stanie Scan,
- wymusza EmergencyStop przez RequestEmergencyStop,
- po dojechaniu do konca moze sie zatrzymac.

Klucz:

[blok kodu / diagram w pliku Markdown]

To nie jest fizyczne zderzenie. To celowe wymuszenie reakcji FSM.

15. HUD i debug

DebugHud.cs pokazuje:

- aktualny stan FSM,
- predkosc auta,
- kat skretu,
- odczyty sensorow,
- czy trwa sledzenie luki,
- ostatniego kandydata miejsca parkingowego.

To pomaga podczas prezentacji, bo widac, ze system faktycznie przechodzi przez stany.

16. Kamera

CameraFollow.cs sledzi auto:

[blok kodu / diagram w pliku Markdown]

Kamera nie jest elementem algorytmu, tylko ulatwia obserwacje.

17. UI

Projekt ma dwa typy UI:

1. Canvas z przyciskami tworzony przez ParkingDemoBuilder.
2. Awaryjne UI OnGUI z ImmediateMapUi.

OnGUI jest malo eleganckie, ale praktyczne: nawet jesli Canvas nie zadzialalby poprawnie, dalej mozna zmieniac mapy.

18. Dlaczego nie ML-Agents

Projekt nie uzywa ML-Agents, bo:

- uczenie modelu zajmuje duzo czasu,
- wynik uczenia nie jest deterministyczny,

- trudniej wyjasnic decyzje auta,
- na projekt zaliczeniowy wystarcza FSM + sensory + scenariusze.

To jest zaleta w dokumentacji: prowadzacy moze zapytac "dlaczego bez ML?", a odpowiedz brzmi: wymagany byl algorytm automatycznego parkowania, niekoniecznie uczony model; zastosowano deterministyczny automat stanow i sensory.

19. Ograniczenia projektu

Najwazniejsze ograniczenia:

- auto nie ma pelnego modelu zawieszenia,
- finalny manewr nie jest obliczany dynamicznie z geometrii miejsca, tylko zapisany punktami,
- trasy sa dopasowane do przygotowanych scen,
- kolizje nie sa glownym mechanizmem sterowania,
- projekt pokazuje zasade dzialania, a nie gotowy system do prawdziwego auta.

Te ograniczenia sa akceptowalne, jesli sa uczciwie opisane.

20. Co mowic przy prezentacji

Krotkie wytлумaczenie:

"Projekt pokazuje automatyczne parkowanie zrealizowane deterministycznie. Auto ma sensory Raycast, ktore mierza odleglosci od przeszkod, a logika jest oparta o maszyny stanow. Po znalezieniu miejsca auto wykonuje manewr wedlug zaprogramowanej trajektorii punktowej. W trzeciej scenie pojawia sie auto z naprzeciwka, ktore wymusza stan EmergencyStop. Rozwiazanie nie uzywa ML, bo celem byla powtarzalna demonstracja algorytmu i reakcji na przeszkody."

02_Jak_stworzyc_od_zera

Jak stworzyc taki projekt od zera

Ten dokument opisuje proces stworzenia podobnego projektu w Unity krok po kroku. Nie jest to tylko instrukcja uruchomienia gotowej paczki, ale opis jak samodzielnie zbudowac podobna symulacje.

1. Przygotowanie projektu Unity

1. Otworz Unity Hub.
2. Utworz nowy projekt typu Universal 3D.
3. Nazwij projekt np. Automatic parking.
4. Po otwarciu Unity utworz foldery:

[blok kodu / diagram w pliku Markdown]

Minimalnie potrzebne sa:

- Scripts/Parking na skrypty runtime,
- Editor na generator scen,
- Scenes na sceny,
- Materials na materiały.

2. Podstawowe obiekty w Unity

Projekt można zrobić bez gotowych assetów, tylko z prymitywów Unity:

- Cube jako droga,
- Cube jako samochód,
- Cube jako kabina,
- Cylinder jako koła,
- Cube jako linie parkingowe,
- Cube jako przeszkody i ściany.

To wystarczy do zaliczenia, bo najważniejsza jest logika.

3. Stworzenie typów danych

Pierwszy plik to ParkingTypes.cs.

Należy zdefiniować:

- enum stanów FSM,
- enum typu parkowania,
- struktury odczytów sensorów,
- struktury kandydata miejsca.

Przykład stanów:

[blok kodu / diagram w pliku Markdown]

Dodatkowy stan:

[blok kodu / diagram w pliku Markdown]

Ta separacja jest ważna, bo łatwiej potem tłumaczyć kod.

4. Stworzenie kontrolera auta

Plik: ParkingCarController.cs

Kontroler powinien mieć pola:

- wheelBase,
- carLength,
- carWidth,
- maxForwardSpeed,
- maxReverseSpeed,
- acceleration,
- brakeAcceleration,
- maxSteeringAngle,
- steeringRate.

Minimalne metody:

[blok kodu / diagram w pliku Markdown]

Tryb pierwszy: kinematyczny

W prostym trybie auto jedzie tak:

1. Oblicz prędkość docelową z gazu.
2. Płynnie zmieniaj aktualną prędkość.
3. Płynnie zmieniaj kąt skrętu.
4. Oblicz prędkość obrotową:

[blok kodu / diagram w pliku Markdown]

5. Przesun auto do przodu:

[blok kodu / diagram w pliku Markdown]

Tryb drugi: ruch po punktach

Do prezentacji najlepiej użyć ruchu po punktach:

1. Zapamiętaj pozycję startową odcinka.
2. Zapamiętaj kąt startowy.

3. Oblicz różnicę kąta przez `Mathf.DeltaAngle`.
4. Przesuwaj auto przez `Vector3.Lerp`.
5. Obracaj auto przez łagodne przejście `SmoothStep`.

To daje stabilny pokaz bez losowej fizyki.

5. Stworzenie sensorów

Plik: `ParkingSensors.cs`

Sensory powinny mierzyć:

- przód,
- tył,
- bok prawy: przód/srodek/tył,
- bok lewy: przód/srodek/tył.

W Unity robi się to przez:

[blok kodu / diagram w pliku Markdown]

Warto rysować promienie:

[blok kodu / diagram w pliku Markdown]

Kolory:

- zielony: wolne,
- czerwony: przeszkoda.

6. Wykrywanie miejsca

Prosta logika:

1. Jeśli trzy sensory boczne pokazują dużą odległość, zaczyna się luka.
2. Zapamiętaj pozycję początku luki.
3. Gdy auto jedzie dalej, aktualizuj koniec luki.
4. Jeśli luka jest dłuższa niż wymagane minimum, sprawdź objętość `CheckBox`.
5. Jeśli `CheckBox` nie wykrywa przeszkody, miejsce jest poprawne.

Dla parkowania równoległego wymagaj dłuższej luki, np.:

[blok kodu / diagram w pliku Markdown]

Dla prostopadłego:

[blok kodu / diagram w pliku Markdown]

7. Stworzenie FSM

Plik: ParkingStateMachine.cs

Najprostszy wzorzec:

[blok kodu / diagram w pliku Markdown]

W każdym stanie:

- wykonaj logikę,
- sprawdź warunek przejścia,
- jeśli warunek spełniony, wywołaj `ChangeState(next)`.

Metoda `ChangeState` powinna:

- ustawić nowy stan,
- wyzerować licznik czasu stanu,
- wykonać akcje specjalne, np. zatrzymanie w `Parked`.

8. Dodanie `EmergencyStop`

Stan awaryjny powinien działać z każdego etapu manewru.

Przykład:

[blok kodu / diagram w pliku Markdown]

W projekcie przeszkoda dynamiczna może też wywołać:

[blok kodu / diagram w pliku Markdown]

To pozwala pokazać reakcje nawet wtedy, gdy fizyczne kolizje są uproszczone.

9. Stworzenie scenariusza manewru

Plik: ParkingScenario.cs

Najprostszy sposob:

1. Utworz liste punktow docelowych.
2. Kazdy punkt ma:
 - pozycje,
 - kat,
 - predkosc,
 - informacje czy jedziemy do przodu czy cofamy,
 - stan FSM, ktory ma byc wyswietlany.
3. FSM wykonuje punkty po kolei.

Przyklad:

[blok kodu / diagram w pliku Markdown]

Im wiecej malych punktow, tym plynniej wyglada ruch.

10. Jak stroic punkty parkowania

Najpraktyczniejsza metoda:

1. Ustaw auto przed miejscem.
2. Dodaj punkt pozycjonowania.
3. Dodaj punkt wejscia w miejsce.
4. Dodaj punkt polowy.
5. Dodaj punkt koncowy.
6. Uruchom Play.
7. Jesli auto zahacza:
 - odsun punkty od przeszkody,
 - zmniejsz skok kata miedzy punktami,
 - dodaj punkt posredni.
8. Jesli auto stoi na linii:
 - przesun finalny punkt do srodka miejsca.

Dla miejsca miedzy liniami zA i zB srodek to:

[blok kodu / diagram w pliku Markdown]

Przyklad z mapy 3:

[blok kodu / diagram w pliku Markdown]

11. Generator scen

Zamiast ręcznie klikać wszystko w Unity, można napisać skrypt edytorski:

[blok kodu / diagram w pliku Markdown]

Taki generator:

- tworzy materiały,
- tworzy auta,
- tworzy drogi,
- tworzy linie,
- ustawia UI,
- ustawia kamerę,
- zapisuje sceny.

Zaletą: jeśli coś się zepsuje, można jednym kliknięciem odbudować cały projekt.

12. Stworzenie modelu auta

Minimalny model:

1. Body - cube, skala np. 2.0 x 0.72 x 4.45.
2. Cabin - mniejszy cube na gorze.
3. FrontMarker - mały pasek z przodu.
4. Cztery koła - cylindry.

Do auta dodaj:

- Rigidbody jako kinematic,
- ParkingCarController,
- ParkingSensors,
- ParkingStateMachine,
- ParkingManeuverPlanner,
- VehicleVisualRig.

13. Stworzenie mapy 1

Mapa 1:

- droga na środku,
- parking po prawej,

- wiekszosc miejsc zajeta,
- jedna luka za waska,
- jedna luka docelowa.

Cel: pokazac, ze auto nie parkuje w pierwszym lepszym miejscu.

14. Stworzenie mapy 2

Mapa 2:

- ulica,
- auta zaparkowane rownolegle,
- miejsce docelowe,
- manewr cofania.

Cel: pokazac parkowanie rownolegle.

15. Stworzenie mapy 3

Mapa 3:

- parking po lewej,
- czerwone auto z naprzeciwka,
- niebieskie auto czeka,
- po przejechaniu czerwonego auta parkuje.

Cel: pokazac stan EmergencyStop.

16. Dodanie HUD

HUD powinien pokazywac:

- aktualny stan,
- predkosc,
- kat skretu,
- odczyty sensorow,
- czy jest kandydat miejsca.

To bardzo pomaga podczas obrony.

17. Testowanie

Testuj w tej kolejnosci:

1. Czy sceny sie generuja.
2. Czy menu laduje mapy.
3. Czy auto rusza.
4. Czy HUD pokazuje zmiane stanow.

5. Czy auto konczy w Parked.
6. Czy mapa 3 przechodzi przez EmergencyStop.
7. Czy finalne pozycje aut nie zahaczaja innych aut.

18. Build

Po wygenerowaniu scen:

1. Otworz File -> Build Profiles albo File -> Build Settings.
2. Sprawdz, czy sceny sa w liscie:
 - MainMenu,
 - Map_01_Perpendicular,
 - Map_02_Parallel,
 - Map_03_Dynamic.
3. Wybierz Windows.
4. Zbuduj .exe.

19. Co nagrać na filmie

Film 2-3 min:

1. Menu.
2. Mapa 1: auto parkuje prostopadle.
3. Mapa 2: auto parkuje rownolegle.
4. Mapa 3: czerwone auto wymusza zatrzymanie, potem niebieskie parkuje po lewej.
5. Krotko pokaz Scene View z promieniami sensorow.

20. Jak tłumaczyc uproszczenia

Najlepsze wyjasnienie:

"Projekt skupia sie na algorytmie sterowania, a nie na zaawansowanej fizyce pojazdu. Dlatego zastosowano uproszczony model kinematyczny i deterministyczny tor parkowania po punktach. Dzieki temu wynik jest powtarzalny, a w projekcie widac sensory, FSM i reakcje na przeszkody."

03_Testy_i_uruchomienie

Testy, uruchomienie i kryteria zaliczenia

1. Instalacja paczki

Zalecana metoda:

1. Zamknij Unity.

2. Wypakuj paczkę.
3. Uruchom Install-ToUnityProject.ps1.
4. Otwórz Unity.
5. Poczekaj na kompilację.
6. Kliknij:

[blok kodu / diagram w pliku Markdown]

7. Otwórz:

[blok kodu / diagram w pliku Markdown]

8. Uruchom Play.

2. Dlaczego trzeba kliknąć Build Demo Scenes

Skrypty same nie zmieniają istniejących scen. Generator musi:

- odbudować pozycje aut,
- odbudować punkty trajektorii,
- odbudować UI,
- zapisać sceny.

Jeśli po instalacji nie klikniesz Build Demo Scenes, Unity może nadal uruchamiać starą wersję mapy.

3. Test ogólny

Po starcie projektu sprawdź:

- czy menu ma trzy przyciski,
- czy każda mapa się ładuje,
- czy widac HUD w lewym górnym rogu,
- czy auto nie wypadnie poza drogę,
- czy po zaparkowaniu HUD pokazuje Parked.

4. Test mapy 1

Nazwa sceny:

[blok kodu / diagram w pliku Markdown]

Cel:

- parking prostopadly,
- auto ma zaparkowac w miejscu docelowym,
- za waska luka ma byc tylko elementem demonstracyjnym.

Oczekiwany przebieg:

1. Auto startuje z dolu mapy.
2. Jedzie wzdluz drogi.
3. HUD pokazuje Scan.
4. Po dojechaniu do punktu akceptacji przechodzi do ValidateSpot.
5. Potem przechodzi przez:
 - Positioning,
 - ReverseTurn,
 - CounterTurn,
 - Straighen.
6. Konczy w Parked.

Co sprawdzic wizualnie:

- auto stoi w miejscu docelowym,
- nie zatrzymuje sie w za waskiej luce,
- nie obraca sie w miejscu,
- nie zahacza zaparkowanych aut.

5. Test mapy 2

Nazwa sceny:

[blok kodu / diagram w pliku Markdown]

Cel:

- parkowanie rownolegle,
- manewr cofania i kontry.

Oczekiwany przebieg:

1. Auto jedzie ulica.
2. Dojezdza do punktu akceptacji.
3. Cofa po kilku punktach.
4. Wykonuje kontre.
5. Prostuje sie.
6. Konczy w Parked.

Co sprawdzić:

- przód auta nie uderza w auto przed nim,
- auto nie kończy krzywo,
- finalnie stoi równoległe do drogi.

6. Test mapy 3

Nazwa sceny:

[blok kodu / diagram w pliku Markdown]

Cel:

- parking po lewej stronie,
- czerwone auto jedzie z naprzeciwka,
- niebieskie auto ma zaczekać,
- potem ma zaparkować po lewej.

Oczekiwany przebieg:

1. Niebieskie auto rusza.
2. Czerwone auto jedzie z naprzeciwka.
3. Niebieskie auto przechodzi do EmergencyStop.
4. Czerwone auto przejeżdża.
5. Niebieskie auto kontynuuje.
6. Niebieskie auto parkuje po lewej.
7. Kończy w Parked.

Finalne miejsce:

- linie miejsca są przy $z = -10.1$ oraz $z = -6.7$,
- środek miejsca to $z = -8.4$,
- finalny punkt auta to $(-5.62, 0, -8.4)$, kąt 90° .

Co sprawdzić:

- niebieskie auto nie stoi na linii,
- niebieskie auto nie zahacza auta zaparkowanego obok,
- czerwone auto nie zderza się z niebieskim,
- po zatrzymaniu auto kontynuuje manewr.

7. Test sensorów

W Scene View powinno być widac promienie:

- zielone, gdy nie ma przeszkody,
- czerwone, gdy sensor trafia przeszkodę.

Sprawdz:

- sensor przedni,
- sensor tylny,
- trzy sensory po prawej,
- trzy sensory po lewej.

8. Test HUD

HUD powinien pokazywać:

[blok kodu / diagram w pliku Markdown]

Jesli HUD jest pusty:

- sprawdź, czy DebugHud ma przypisane referencje,
- wygeneruj sceny jeszcze raz,
- sprawdź, czy auto ma komponenty ParkingStateMachine, ParkingCarController, ParkingSensors.

9. Typowe problemy i naprawy

Auto dalej zachowuje się jak w starej wersji

Przyczyna:

- scena nie została przebudowana.

Naprawa:

[blok kodu / diagram w pliku Markdown]

Nie ma menu Tools

Możliwe przyczyny:

- plik ParkingDemoBuilder.cs nie jest w Assets/Editor,
- Unity jeszcze kompiluje skrypty,
- w konsoli są błędy kompilacji.

Przyciski UI nie działają

Projekt ma zapasowe UI OnGUI. W lewym górnym rogu powinny być przyciski Menu, Restart, Map 1, Map 2, Map 3.

Auto stoi na linii

Trzeba przesunąć finalny punkt scenariusza. Dla miejsca ograniczonego liniami:

[blok kodu / diagram w pliku Markdown]

Auto zahacza inne auto

Trzeba:

- odsunąć punkty pośrednie od przeszkody,
- dodać więcej małych kroków,
- zmniejszyć zmianę kąta między punktami,
- sprawdzić finalny punkt.

10. Kryteria zaliczenia

Projekt spełnia założenia, jeżeli:

- ma 3 sceny,
- auto porusza się automatycznie,
- jest FSM,
- są czujniki Raycast,
- widac reakcje na przeszkodę dynamiczną,
- jest UI do zmiany map,
- jest dokumentacja,
- da się zbudować .exe,
- logika nie korzysta z ML-Agents.

11. Lista kontrolna przed oddaniem

Przed oddaniem sprawdź:

- [] Wpisane imię i nazwisko w README.
- [] Wpisany numer albumu.
- [] Sceny wygenerowane przez Build Demo Scenes.
- [] Mapa 1 działa.
- [] Mapa 2 działa.
- [] Mapa 3 działa.
- [] Zrobiony build Windows.
- [] Nagrany film 2-3 min.
- [] W filmie widac HUD.
- [] W filmie widac trzy mapy.

04_Opis_do_obrony

Opis do obrony projektu

Ten dokument jest krótka ściągą do rozmowy z prowadzącym.

1. Jednozdaniowy opis projektu

Projekt przedstawia deterministyczną symulację automatycznego parkowania w Unity, w której auto wykrywa miejsca parkingowe sensorami Raycast, przechodzi przez maszyny stanów FSM i wykonuje zaprogramowany manewr parkowania.

2. Co jest najważniejsze technicznie

Najważniejsze elementy:

- Unity 3D jako środowisko symulacyjne,
- C# i MonoBehaviour,
- sensory Physics.Raycast,
- walidacja miejsca przez Physics.CheckBox,
- FSM jako główny algorytm decyzyjny,
- deterministyczny tor parkowania po punktach,
- przeszkoda dynamiczna wymuszająca EmergencyStop,
- HUD debug pokazujący stan algorytmu.

3. Dlaczego projekt nie używa ML

Mozna powiedzieć:

"Nie zastosowałem ML-Agents, ponieważ celem projektu było pokazanie algorytmu automatycznego parkowania, a nie procesu uczenia modelu. Rozwiązanie deterministyczne jest łatwiejsze do wytłumaczenia, powtarzalne i stabilne podczas prezentacji. W projekcie widac sensory, walidację miejsca, automat stanów i reakcje na przeszkodę."

4. Jak działa auto

Auto ma dwa poziomy sterowania:

1. Podczas dojazdu jedzie kinematycznie.
2. Po znalezieniu miejsca wykonuje manewr po punktach trajektorii.

Powód:

- pełna fizyka auta wymagałaby strojenia zawieszenia, tarcia i masy,

- w projekcie zaliczeniowym ważniejszy jest algorytm i powtarzalny pokaz.

5. Jak działają sensory

Sensory są promieniami:

- jeden z przodu,
- jeden z tyłu,
- trzy z prawej,
- trzy z lewej.

Promienie zwracają odległość do najbliższej przeszkody. Jeżeli z boku jest wolna przestrzeń przez odpowiedni dystans, program uznaje, że może tam być luka parkingowa.

Dodatkowo CheckBox sprawdza, czy w obszarze miejsca nie ma przeszkody.

6. Jak działa FSM

FSM przechodzi przez stany:

[blok kodu / diagram w pliku Markdown]

Znaczenie:

- Scan - szukanie miejsca,
- ValidateSpot - potwierdzenie miejsca,
- Positioning - ustawienie auta,
- ReverseTurn - pierwszy skręt,
- CounterTurn - kontra,
- Straighten - wyprostowanie,
- Parked - koniec.

Jest też EmergencyStop, gdy pojawia się przeszkoda.

7. Co pokazać na mapie 1

Powiedz:

"Mapa 1 pokazuje parkowanie prostopadłe. Większość miejsc jest zajęta, jest też za wąska luka. Auto finalnie parkuje w poprawnym miejscu."

Pokaż:

- HUD,
- przejścia stanów,
- finalne zaparkowanie.

8. Co pokazac na mapie 2

Powiedz:

"Mapa 2 pokazuje parkowanie rownolegle. Auto wykonuje cofanie, kontre i prostowanie."

Pokaz:

- cofanie,
- ustawienie rownolegle,
- stan Parked.

9. Co pokazac na mapie 3

Powiedz:

"Mapa 3 pokazuje reakcje na ruchoma przeszkode. Czerwone auto jedzie z naprzeciwka, dlatego niebieskie auto zatrzymuje sie w EmergencyStop. Po przejechaniu przeszkody kontynuuje i parkuje po lewej stronie."

Pokaz:

- czerwone auto,
- zatrzymanie niebieskiego,
- kontynuacje manewru,
- finalne parkowanie po lewej.

10. Pytania i odpowiedzi

Czy to jest prawdziwa fizyka auta?

Nie w pelnym sensie. To uproszczony model kinematyczny i manewr po punktach. Pelna fizyka auta w Unity wymagalaby dokladnego strojenia WheelCollider, masy, tarcia i zawieszenia.

Czy auto samo znajduje miejsce?

W projekcie sa sensory i struktura kandydata miejsca. W scenach demonstracyjnych miejsce jest dodatkowo opisane w ParkingScenario, zeby pokaz byl powtarzalny.

Czy mozna dodac inne mapy?

Tak. Trzeba dodac nowa scene albo rozbudowac ParkingDemoBuilder, ustawic auta, linie i nowe punkty ParkingScenarioStep.

Co jest najwiekszym uproszczeniem?

Najwieksze uproszczenie to finalny manewr parkowania po zaprogramowanych punktach, zamiast

dynamicznego wyznaczania trajektorii w czasie rzeczywistym.

Co jest najważniejszym elementem algorytmu?

Maszyna stanów FSM i sensory. FSM decyduje, co auto robi w danym momencie, a sensory dostarczają informacji o przeszkodach i miejscu.

11. Co powiedzieć o własnym wkładzie

Mozna powiedzieć:

"Projekt został zrobiony jako indywidualna implementacja w Unity. Logika sterowania, FSM, sensory, scenariusze, UI i dokumentacja zostały przygotowane od zera. Nie korzystam z ML-Agents ani gotowego modelu uczonego."

12. Krótki opis do sprawozdania

Projekt przedstawia symulację automatycznego parkowania samochodu w Unity 3D. System opiera się na maszynie stanów FSM oraz czujnikach wirtualnych wykonanych za pomocą Physics.Raycast. Czujniki mierzą odległości od przeszkód z przodu, z tyłu oraz po bokach pojazdu. Po wykryciu poprawnego miejsca parkingowego auto przechodzi przez kolejne stany manewru: ustawienie, skręt, kontre, prostowanie i zatrzymanie. W scenie dynamicznej czerwony pojazd nadjeżdżający z naprzeciwka wymusza zatrzymanie awaryjne. Dla stabilności demonstracji finalny manewr parkowania jest realizowany po wyznaczonych punktach trajektorii, co zapewnia powtarzalny wynik i ułatwia prezentację algorytmu.

05_Mapa_plikow_i_parametrow

Mapa plików i parametrów

Ten dokument pomaga szybko znaleźć, gdzie w kodzie zmienia się konkretne rzeczy.

1. Gdzie zmienić prędkość auta

Plik:

[blok kodu / diagram w pliku Markdown]

Pola:

[blok kodu / diagram w pliku Markdown]

W scenach generator ustawia:

[blok kodu / diagram w pliku Markdown]

2. Gdzie zmienić skręt auta

Plik:

[blok kodu / diagram w pliku Markdown]

Pola:

[blok kodu / diagram w pliku Markdown]

maxSteeringAngle to maksymalny kąt skretu. steeringRate to szybkość zmiany skretu.

3. Gdzie zmienić sensory

Plik:

[blok kodu / diagram w pliku Markdown]

Pola:

[blok kodu / diagram w pliku Markdown]

Najważniejsze:

- forwardRange - zasięg przód/tył,
- sideRange - zasięg boczny,
- freeSideThreshold - odległość uznawana za wolną przestrzeń,
- parallelRequiredLength - minimalna długość luki równoległej,
- perpendicularRequiredWidth - minimalna szerokość luki prostopadłej.

4. Gdzie zmienić stany FSM

Plik:

[blok kodu / diagram w pliku Markdown]

Enum:

[blok kodu / diagram w pliku Markdown]

Logika przejść:

[blok kodu / diagram w pliku Markdown]

5. Gdzie zmienić tor parkowania mapy 1

Plik:

[blok kodu / diagram w pliku Markdown]

Metoda:

[blok kodu / diagram w pliku Markdown]

Lista:

[blok kodu / diagram w pliku Markdown]

Każdy Step ma:

[blok kodu / diagram w pliku Markdown]

6. Gdzie zmienić tor parkowania mapy 2

Plik:

[blok kodu / diagram w pliku Markdown]

Metoda:

[blok kodu / diagram w pliku Markdown]

7. Gdzie zmienić tor parkowania mapy 3

Plik:

[blok kodu / diagram w pliku Markdown]

Metoda:

[blok kodu / diagram w pliku Markdown]

Finalny punkt aktualnie:

[blok kodu / diagram w pliku Markdown]

Znaczenie:

- $x = -5.62$ - głębokość w miejscu po lewej stronie,
- $z = -8.4$ - środek miejsca między liniami,
- $\text{yaw} = 90$ - auto stoi prostopadle do drogi.

8. Gdzie zmienić czerwone auto w mapie 3

Plik:

[blok kodu / diagram w pliku Markdown]

Metoda:

[blok kodu / diagram w pliku Markdown]

Punkty:

[blok kodu / diagram w pliku Markdown]

Predkosc:

[blok kodu / diagram w pliku Markdown]

9. Gdzie zmienic wyglad auta

Plik:

[blok kodu / diagram w pliku Markdown]

Metoda:

[blok kodu / diagram w pliku Markdown]

Tam sa:

- Body,
- Cabin,
- FrontLights,
- kola.

10. Gdzie zmienic kolory

Plik:

[blok kodu / diagram w pliku Markdown]

Metoda:

[blok kodu / diagram w pliku Markdown]

Przykłady:

[blok kodu / diagram w pliku Markdown]

11. Gdzie zmienić kamerę

Plik:

[blok kodu / diagram w pliku Markdown]

Pola:

[blok kodu / diagram w pliku Markdown]

12. Gdzie zmienić UI

Pliki:

[blok kodu / diagram w pliku Markdown]

MapManager ładuje sceny. ImmediateMapUi rysuje awaryjne przyciski. ParkingDemoBuilder tworzy Canvas.

13. Gdzie zmienić HUD

Plik:

[blok kodu / diagram w pliku Markdown]

Tekst HUD powstaje w:

[blok kodu / diagram w pliku Markdown]

Mozna tam dodać np. aktualny indeks punktu, ale teraz jest prywatny w ParkingStateMachine.

14. Najważniejsze parametry mapy 3

Zaparkowane auta po lewej:

[blok kodu / diagram w pliku Markdown]

Miejsce docelowe:

[blok kodu / diagram w pliku Markdown]

Finalny punkt auta:

[blok kodu / diagram w pliku Markdown]

15. Szybka sciaga: co zmienic gdy cos wyglada zle

Problem Co zmienic

--- ---

Auto stoi na linii finalne z w ostatnim punkcie scenariusza

Auto zahacza inne auto punkty posrednie przed finalnym punktem

Auto obraca sie za mocno zmniejszyc roznice katow miedzy punktami

Auto jedzie za szybko speed w Step(...)

Czerwone auto za pozno wyjezdza triggerDelay albo speed w CreateOncomingTrafficCar

HUD zaslania ekran pozycje CreateText w CreateHud